SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal		
Instructor(s) Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju		
CourseCode	23CS002 PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week7 – Wednusday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All batches
Assignment Number:13.3(Present assignment number)/**24**(Total number of			

assignments)		
	Question	***Expected Time to complete***
	Lab 13: Code Refactoring Using AI Assistance Improving Legacy Code for Readability, Maintainability, and Performance **Lab Objectives** The objectives of this laboratory exercise are to: • Identify common code smells and inefficiencies in legacy Python programs. • Use AI-assisted coding tools to refactor code for better readability and maintainability. • Apply modern Python best practices while preserving original program behavior. • Analyze performance improvements achieved through refactoring.	
1	**Learning Outcomes** After completing this lab, students will be able to: • Detect and eliminate code duplication in Python programs. • Refactor inefficient logic using optimized data structures and constructs. • Improve code quality through meaningful naming conventions and documentation. • Validate refactored code using test cases and performance comparison.	Week7 - Wednesday

	Task 1: Refactoring – Removing Code Duplication
	Objective
	To eliminate repeated logic by extracting reusable functions.
	Task Description
	Use AI assistance to refactor a legacy Python script that contains repeated blocks of code calculating the area and perimeter of rectangles.
	Starter (Legacy) Code
	# Legacy script with repeated logic
	print("Area of Rectangle:", 5 * 10)
	print("Perimeter of Rectangle:", 2 * (5 + 10))
	print("Area of Rectangle:", 7 * 12)
	print("Perimeter of Rectangle:", 2 * (7 + 12))	

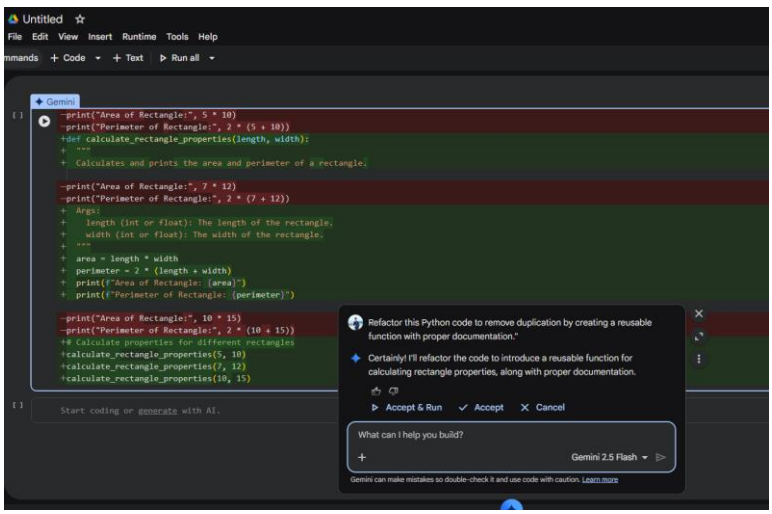
```
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

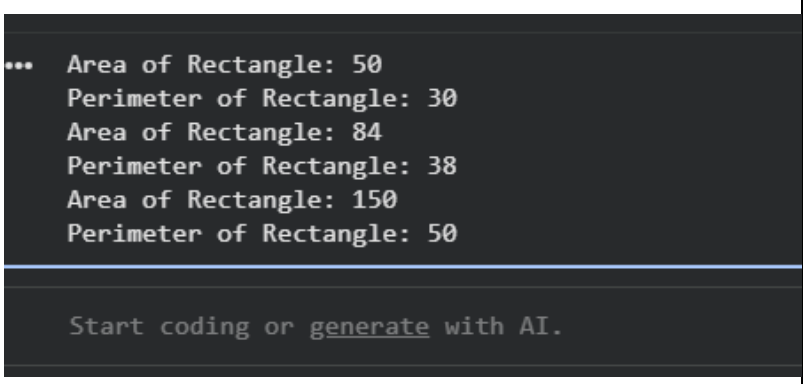
Expected Outcome

- A reusable function to calculate area and perimeter
- No duplicated code blocks
- Proper docstrings for all functions

AI Prompt Used

"Refactor this Python code to remove duplication by creating a reusable function with proper documentation."





The repeated calculations were replaced with a reusable function called **rectangle_properties()**.

This improves code readability and maintainability by avoiding duplicated code.

Task 2: Refactoring – Optimizing Loops and Conditionals
Objective
To improve performance by replacing inefficient nested loops with optimized structures.
Task Description
Use AI to analyze and refactor a script that checks the presence of elements using nested loops.
Starter (Legacy) Code

```
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
found = False
for n in names:
if s == n:
found = True
if found:
print(f"{s} is in the list")
else:
print(f"{s} is not in the list")
```

Expected Outcome

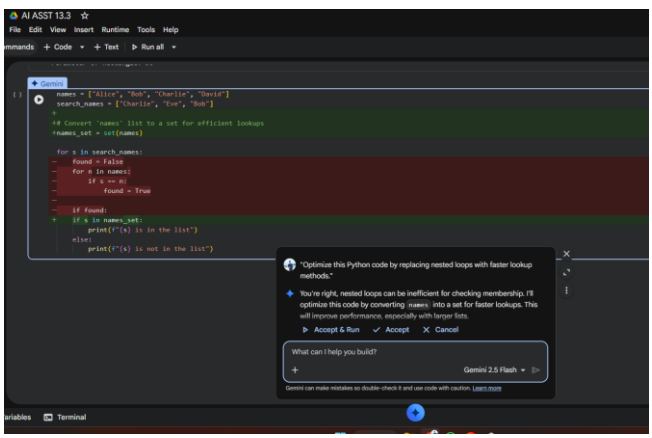
- Optimized solution using set lookups or comprehensions
- Performance comparison before and after refactoring

```python
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True

    if found:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

"Optimize this Python code by replacing nested loops with faster lookup methods."




```
Charlie is in the list
Eve is not in the list
Bob is in the list
```

The nested loop was replaced with **set lookup**, which has **O(1) average time complexity**, improving performance compared to **O(n^2)** nested loops.

Task 3: Refactoring – Extracting Reusable Functions
Objective
To modularize code by extracting calculations into reusable functions.
Task Description
Refactor a legacy script where price and tax calculations are written inline.
Starter (Legacy) Code

```
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

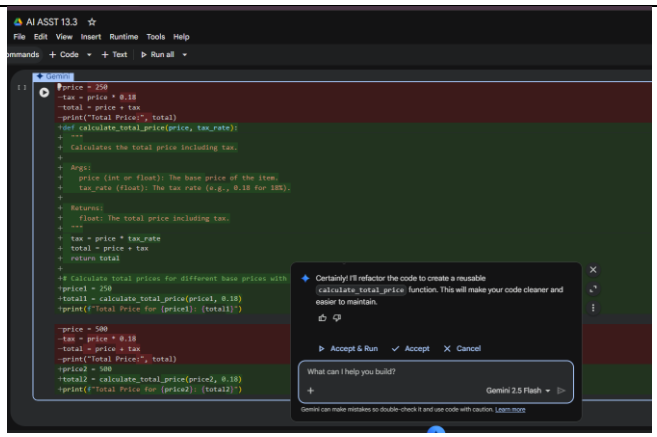
Expected Outcome

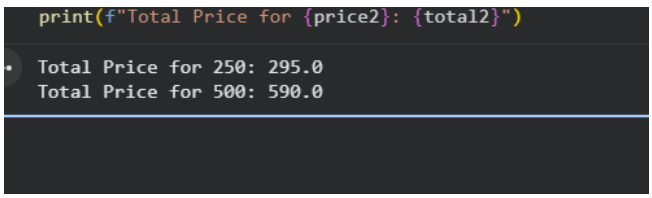
- A function calculate_total(price)
- Cleaner and reusable code structure
- Proper documentation

```
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

"Refactor the code to create a reusable function for calculating total price including tax."





The repeated price calculation was converted into a reusable function **calculate_total()**.
This makes the program modular and easier to maintain.

Task 4: Refactoring – Replacing Hardcoded Values with Constants

Objective

To improve maintainability by replacing magic numbers with named constants.

Task Description

Use AI to identify hardcoded values and replace them with constants.

Starter (Legacy) Code

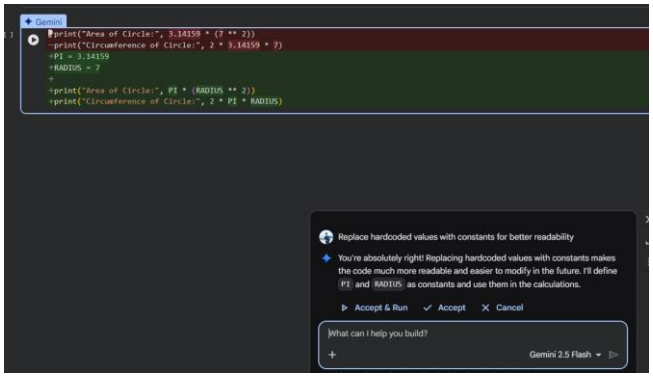
```
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
```

Expected Outcome

- Constants such as PI and RADIUS
- Cleaner, maintainable code

```
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
```

Replace hardcoded values with constants for better readability



Magic numbers were replaced with named constants **PI** and **RADIUS**, improving readability and maintainability.

Task 5: Refactoring – Improving Variable Naming and Readability

Objective

To enhance readability using descriptive variable names and comments.

Task Description

Refactor a script with unclear variable names.

Starter (Legacy) Code

```
a = 10
b = 20
c = a * b / 2
print(c)
```

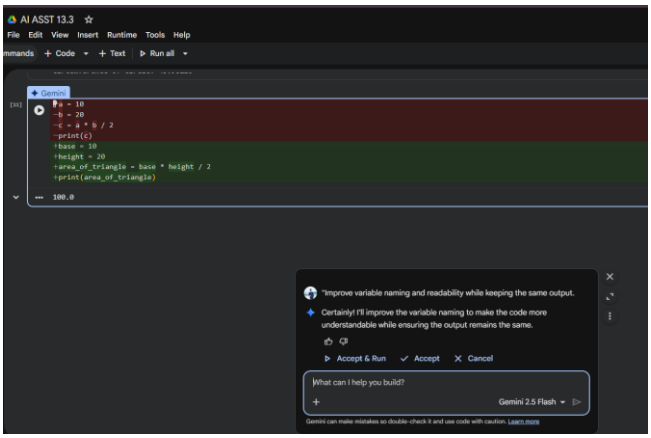
Expected Outcome

- Descriptive variable names
- Inline comments explaining logic
- Identical output

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

```
a = 10
b = 20
c = a * b / 2
print(c)
```

"Improve variable naming and readability while keeping the same output.



The variables were renamed to **base**, **height**, and **area_of_triangle** to make the code more readable and understandable.

After completing this lab:

- Code duplication was eliminated using reusable functions.
- Inefficient nested loops were replaced with optimized data structures.
- Hardcoded values were replaced with constants.
- Variable names and comments improved code readability.
- AI-assisted coding tools helped refactor legacy code efficiently.